

```
import React, { useState, useEffect, useRef, useMemo, useCallback } from "react";
```

```
/* ===== ПОСТРОЕНИЕ СЕТКИ ===== *
```

```
function parseEntries(text) {  
  const seen = new Set(), out = [];  
  text.split("\n").forEach((line) => {  
    const t = line.trim();  
    if (!t) return;  
    const m = t.match(/^(.*?)\s*[\---:=|\t]\s*(.+)$/);  
    let word = m ? m[1] : t, clue = m ? m[2] : "";  
    const norm = word.toUpperCase().replace(/[\p{L}\p{N}]/gu, "");  
    if (norm.length < 2 || seen.has(norm)) return;  
    seen.add(norm);  
    out.push({ word: norm, clue: clue.trim(), raw: word.trim() });  
  });  
  return out;  
}
```

```
function validate(word, sr, sc, dir, get) {  
  const dr = dir === "D" ? 1 : 0, dc = dir === "A" ? 1 : 0;  
  if (get(sr - dr, sc - dc) !== null) return null;  
  if (get(sr + dr * word.length, sc + dc * word.length) !== null) return null;  
  let cr = 0;  
  for (let k = 0; k < word.length; k++) {  
    const r = sr + dr * k, c = sc + dc * k, ex = get(r, c);  
    if (ex !== null) {  
      if (ex !== word[k]) return null;  
      cr++;  
    } else if (dir === "A") {  
      if (get(r - 1, c) !== null || get(r + 1, c) !== null) return null;  
    } else {  
      if (get(r, c - 1) !== null || get(r, c + 1) !== null) return null;  
    }  
  }  
  return cr < 1 ? null : cr;  
}
```

```
function bboxArea(cells, extra) {  
  let a = Infinity, b = Infinity, x = -Infinity, y = -Infinity;  
  const u = (r, c) => { if (r < a) a = r; if (c < b) b = c; if (r > x) x = r; if  
  cells.forEach((_, key) => { const [r, c] = key.split(",").map(Number); u(r, c);  
  if (extra) extra.forEach(([r, c]) => u(r, c));  
  return (x - a + 1) * (y - b + 1);  
}
```

```
}
```

```
function shuffle(arr) {  
  const a = arr.slice();  
  for (let i = a.length - 1; i > 0; i--) {  
    const j = Math.floor(Math.random() * (i + 1));  
    [a[i], a[j]] = [a[j], a[i]];  
  }  
  return a;  
}
```

```
function attempt(entries) {  
  const cells = new Map();  
  const get = (r, c) => cells.get(r + "," + c);  
  const placements = [];  
  const order = shuffle(entries).sort((a, b) => b.word.length - a.word.length);  
  const first = order[0];  
  for (let k = 0; k < first.word.length; k++) cells.set("0," + k, first.word[k]);  
  placements.push({ ...first, r: 0, c: 0, dir: "A" });  
  
  let remaining = order.slice(1), progress = true;  
  while (progress && remaining.length) {  
    progress = false;  
    const still = [];  
    for (const e of remaining) {  
      const cand = [];  
      for (const p of placements) {  
        for (let i = 0; i < p.word.length; i++) {  
          for (let j = 0; j < e.word.length; j++) {  
            if (p.word[i] !== e.word[j]) continue;  
            let sr, sc, dir;  
            if (p.dir === "A") { dir = "D"; sr = p.r - j; sc = p.c + i; }  
            else { dir = "A"; sr = p.r + i; sc = p.c - j; }  
            const cr = validate(e.word, sr, sc, dir, get);  
            if (cr !== null) {  
              const extra = [], dr = dir === "D" ? 1 : 0, dc = dir === "A" ? 1 :  
                0;  
              for (let k = 0; k < e.word.length; k++) extra.push([sr + dr * k, sc + dc * k]);  
              cand.push({ sr, sc, dir, cr, area: bboxArea(cells, extra) });  
            }  
          }  
        }  
      }  
      if (!cand.length) { still.push(e); continue; }  
      cand.sort((a, b) => (b.cr - a.cr) || (a.area - b.area) || (Math.random() - 0.5));  
      const best = cand[0], dr = best.dir === "D" ? 1 : 0, dc = best.dir === "A" ? 1 : 0;  
      for (let k = 0; k < e.word.length; k++) cells.set((best.sr + dr * k) + "," + (best.sc + dc * k), e.word[k]);  
      placements.push({ ...e, r: best.sr, c: best.sc, dir: best.dir });  
    }  
  }  
}
```

```

        progress = true;
    }
    remaining = still;
}
return { cells, placements, unplaced: remaining };
}

function readWord(sol, r, c, dir) {
    const dr = dir === "D" ? 1 : 0, dc = dir === "A" ? 1 : 0;
    let s = "", rr = r, cc = c;
    while (sol[rr] && sol[rr][cc] !== null) { s += sol[rr][cc]; rr += dr; cc += dc; }
    return s;
}

function finalize(res) {
    let a = Infinity, b = Infinity, x = -Infinity, y = -Infinity;
    res.cells.forEach((_, key) => {
        const [r, c] = key.split(",").map(Number);
        if (r < a) a = r; if (c < b) b = c; if (r > x) x = r; if (c > y) y = c;
    });
    const rows = x - a + 1, cols = y - b + 1;
    const sol = Array.from({ length: rows }, () => Array(cols).fill(null));
    res.cells.forEach((v, key) => { const [r, c] = key.split(",").map(Number); sol[
    const startMap = new Map();
    res.placements.forEach((p) => startMap.set((p.r - a) + "," + (p.c - b) + "," +
    const numbers = Array.from({ length: rows }, () => Array(cols).fill(0));
    const across = [], down = [];
    let n = 0;
    for (let r = 0; r < rows; r++) {
        for (let c = 0; c < cols; c++) {
            if (sol[r][c] == null) continue;
            const sA = (c === 0 || sol[r][c - 1] == null) && c + 1 < cols && sol[r][c +
            const sD = (r === 0 || sol[r - 1][c] == null) && r + 1 < rows && sol[r + 1]
            if (sA || sD) { n++; numbers[r][c] = n; }
            if (sA) {
                const p = startMap.get(r + "," + c + ",A");
                across.push({ num: n, clue: p ? p.clue : "", answer: p ? p.word : readWor
            }
            if (sD) {
                const p = startMap.get(r + "," + c + ",D");
                down.push({ num: n, clue: p ? p.clue : "", answer: p ? p.word : readWord(
            }
        }
    }
    return { sol, rows, cols, numbers, across, down };
}

```

```

function buildCrossword(entries) {
  if (!entries.length) return null;
  let best = null;
  const tries = Math.min(120, 24 + entries.length * 4);
  for (let t = 0; t < tries; t++) {
    const res = attempt(entries);
    const placed = res.placements.length;
    const area = bboxArea(res.cells, null);
    if (!best || placed > best.placed || (placed === best.placed && area < best.a
    if (best.placed === entries.length && best.area <= entries.length * 5) break;
  }
  const fin = finalize(best.res);
  fin.unplaced = best.res.unplaced;
  fin.total = entries.length;
  return fin;
}

function wordCells(sol, r, c, dir) {
  if (!sol[r] || sol[r][c] == null) return [];
  const dr = dir === "D" ? 1 : 0, dc = dir === "A" ? 1 : 0;
  let sr = r, sc = c;
  while (sol[sr - dr] && sol[sr - dr][sc - dc] != null) { sr -= dr; sc -= dc; }
  const out = [];
  let rr = sr, cc = sc;
  while (sol[rr] && sol[rr][cc] != null) { out.push(rr + "-" + cc); rr += dr; cc
  return out;
}

const hasWord = (sol, r, c, dir) => wordCells(sol, r, c, dir).length > 1;

/* ===== ТЕМЫ → ПОДТЕМЫ ===== */

const WORD_COUNT = { easy: 15, medium: 30, hard: 50 };
const THEME_COUNT = { easy: 5, medium: 10, hard: 15 };
const RANGES = { easy: [3, 9], medium: [4, 12], hard: [4, 15] };
const LEVEL_RU = { easy: "лёгкий", medium: "средний", hard: "сложный" };
const STYLE = {
  easy: "Слова узнаваемые, подсказки ясные, но не пересказ самого слова.",
  medium: "Слова средней редкости, подсказки не в лоб.",
  hard: "Слова редкие и профессиональные; подсказки через перифраз и ассоциацию."
};

const THEMES = {
  "Автомобили": ["Марки и модели", "Автоспорт и гонки", "Устройство и двигатель",
  "Космос": ["Планеты и спутники", "Звёзды и галактики", "Космонавтика", "Телеско
  "Природа": ["Деревья", "Погода и стихии", "Горы и реки", "Минералы", "Озёра и в
  "Животные": ["Хищники", "Птицы", "Морские обитатели", "Насекомые", "Рептилии",
  "География": ["Столицы", "Моря и океаны", "Горные системы", "Острова", "Пустыни

```

```

"История": ["Древний мир", "Средневековье", "Великие открытия", "Войны", "Древн
"Искусство": ["Живопись", "Скульптура", "Архитектура", "Музеи", "Стили", "Графи
"Музыка": ["Классика", "Инструменты", "Жанры", "Опера и балет", "Рок", "Джаз",
"Кино": ["Жанры", "Профессии кино", "Награды", "Техника съёмки", "Анимация", "С
"Спорт": ["Лёгкая атлетика", "Водные виды", "Зимние виды", "Единоборства", "Игр
"Техника": ["Электроника", "Авиация", "Корабли", "Инструменты", "Роботы", "Гадж
"Кухня": ["Специи и травы", "Десерты", "Напитки", "Кухни мира", "Выпечка", "Мор
"Литература": ["Поэзия", "Жанры прозы", "Мифы и эпос", "Персонажи", "Драматурги
"Наука": ["Химия", "Физика", "Биология", "Математика", "Геология", "Астрономия"
"Мифология": ["Греческая", "Скандинавская", "Египетская", "Славянская", "Римска
"Море": ["Корабли и паруса", "Морские узлы", "Обитатели глубин", "Навигация", "
"Растения": ["Цветы", "Травы", "Кустарники", "Грибы", "Комнатные", "Экзотика",
"Медицина": ["Анатомия", "Болезни", "Лекарства", "Хирургия", "Органы", "Витамин
"Профессии": ["Ремёсла", "Творческие", "Технические", "Морские", "Медицинские",
"Города мира": ["Европа", "Азия", "Америка", "Африка", "Столицы", "Порты", "Дре
"Одежда и мода": ["Верхняя одежда", "Обувь", "Головные уборы", "Ткани", "Аксесс
"Праздники": ["Новый год", "Карнавал", "Свадьба", "Дни рождения", "Зимние", "На
"Языки и письмо": ["Азбуки", "Грамматика", "Части речи", "Лексика", "Письменнос
"Деньги и финансы": ["Валюты", "Биржа", "Кредиты", "Банки", "Налоги", "Инвестиц
"Транспорт": ["Поезда", "Самолёты", "Автобусы", "Метро", "Велосипеды", "Грузови
"Компьютеры": ["Железо", "Программы", "Интернет", "Игры", "Сети", "Файлы", "Без
"Танцы": ["Бальные", "Народные", "Современные", "Латино", "Балет", "Уличные", "
"Религия": ["Христианство", "Ислам", "Буддизм", "Иудаизм", "Храмы", "Праздники"
"Игры и игрушки": ["Настольные", "Карточные", "Детские", "Логические", "Куклы",
"Школа": ["Предметы", "Урок", "Перемена", "Оценки", "Принадлежности", "Учителя"
"Птицы": ["Хищные", "Водоплавающие", "Певчие", "Домашние", "Экзотические", "Лес
"Насекомые": ["Жуки", "Бабочки", "Пчёлы и осы", "Муравьи", "Стрекозы", "Кузнечи
"Деревья": ["Хвойные", "Лиственные", "Плодовые", "Тропические", "Кустарники", "
"Драгоценности": ["Камни", "Металлы", "Украшения", "Огранка", "Жемчуг", "Короны
"Напитки": ["Горячие", "Соки", "Чай и кофе", "Молочные", "Газированные", "Кокте
"Тело человека": ["Скелет", "Мышцы", "Органы", "Голова", "Конечности", "Кровь",
};

const THEME_NAMES = Object.keys(THEMES);
const pickThemes = (n) => shuffle(THEME_NAMES).slice(0, n);

/* ===== ВСТРОЕННАЯ БАЗА СЛОВ =====

const BANK = {
  "Автомобили": [{"ПОРШЕНЬ", "Ходит вверх-вниз в цилиндре"}, {"ГЛУШИТЕЛЬ", "Гасит
"Космос": [{"ГАЛАКТИКА", "Звёздный остров во Вселенной"}, {"ТУМАННОСТЬ", "Облак
"Природа": [{"ЛАВИНА", "Снежный обвал в горах"}, {"ТЕЙЗЕР", "Бьющий горячий ист
"Животные": [{"ВЫХУХОЛЬ", "Редкий зверёк-водолаз"}, {"КОАЛА", "Сумчатый любител
"География": [{"АРХИПЕЛАГ", "Группа островов"}, {"ФЬОРД", "Узкий залив среди ск
"История": [{"ЛЕТОПИСЬ", "Погодная запись событий"}, {"РЫЦАРЬ", "Конный воин Ср
"Искусство": [{"ФРЕСКА", "Роспись по сырой штукатурке"}, {"ОФОРТ", "Гравюра тра
"Музыка": [{"СОНАТА", "Пьеса из нескольких частей"}, {"СКРИПКА", "Смычковый чет
"Кино": [{"МОНТАЖ", "Сборка кадров в фильм"}, {"СЦЕНАРИЙ", "Текст-основа фильма

```

```

"Спорт": [ ["ЭСТАФЕТА", "Командный бег с передачей"], ["РАПИРА", "Колющее фехтов
"Техника": [ ["ТРАНЗИСТОР", "Полупроводниковый усилитель"], ["КОНДЕНСАТОР", "Нак
"Кухня": [ ["ШАФРАН", "Дорогая пряность из рылец"], ["ВЕНЧИК", "Им взбивают яйца
"Литература": [ ["МЕТАФОРА", "Скрытое сравнение"], ["СОНЕТ", "Стих из четырнадцать
"Наука": [ ["МОЛЕКУЛА", "Мельчайшая частица вещества"], ["ГИПОТЕЗА", "Научное пр
"Мифология": [ ["КЕНТАВР", "Полуконь-получеловек"], ["ГИДРА", "Многоголовое чудо
"Море": [ ["ШТУРВАЛ", "Колесо управления судном"], ["КИЛЬ", "Хребет корпуса кора
"Растения": [ ["ПАПОРОТНИК", "Растение со спорами, без цветков"], ["КАКТУС", "Ко
"Медицина": [ ["АРТЕРИЯ", "Сосуд от сердца"], ["ВАКЦИНА", "Препарат для иммуните
"Профессии": [ ["КУЗНЕЦ", "Мастер ковки металла"], ["ЛОЦМАН", "Проводник судов в
"Города мира": [ ["ВЕНЕЦИЯ", "Город на каналах"], ["КАИР", "Столица у пирамид"],
"Одежда и мода": [ ["СМОКИНГ", "Вечерний мужской костюм"], ["КОРСЕТ", "Утягивающ
"Праздники": [ ["КАРНАВАЛ", "Шествие в масках"], ["ГИРЛЯНДА", "Цепочка огней для
"Языки и письмо": [ ["АЛФАВИТ", "Набор букв по порядку"], ["ИЕРОГЛИФ", "Знак-кар
"Деньги и финансы": [ ["ВАЛЮТА", "Денежная единица страны"], ["АКЦИЯ", "Доля в к
"Транспорт": [ ["ЛОКОМОТИВ", "Тянет железнодорожный состав"], ["ВАГОН", "Часть п
"Компьютеры": [ ["ПРОЦЕССОР", "Главный вычислитель ПК"], ["МОНИТОР", "Экран комп
"Танцы": [ ["ВАЛЬС", "Плавный танец на три счёта"], ["ТАНГО", "Страстный парный
"Религия": [ ["ХРАМ", "Здание для богослужения"], ["ИКОНА", "Священное изображен
"Игры и игрушки": [ ["ШАХМАТЫ", "Игра на 64 клетках"], ["ДОМИНО", "Игра костяшка
"Школа": [ ["УРОК", "Школьное занятие"], ["ПЕРЕМЕНА", "Отдых между уроками"], [
"Птицы": [ ["ОРЁЛ", "Гордый хищник высот"], ["СОВА", "Ночная охотница с круглыми
"Насекомые": [ ["БАБОЧКА", "Порхает и опыляет цветы"], ["СТРЕКОЗА", "Хищница с п
"Деревья": [ ["ДУБ", "Могучее дерево с желудями"], ["БЕРЁЗА", "Белоствольная кра
"Драгоценности": [ ["АЛМАЗ", "Твердейший драгоценный камень"], ["ИЗУМРУД", "Зелё
"Напитки": [ ["КОМПОТ", "Напиток из варёных фруктов"], ["КАКАО", "Тёплый шоколад
"Тело человека": [ ["СКЕЛЕТ", "Костная опора тела"], ["ПОЗВОНОЧНИК", "Гибкий сто
};

```

```

function bankPool(themes) {
  const out = [];
  themes.forEach((th) => { (BANK[th] || []).forEach((w, c) => out.push({ word:
  const seen = new Set();
  return shuffle(out).filter((e) => { if (seen.has(e.word)) return false; seen.ad
}

```

```

const INK_OPTIONS = [
  { name: "Кит. красный", val: "#b81d1d" },
  { name: "Чёрный", val: "#1f1f24" },
  { name: "Синий", val: "#2456c8" },
  { name: "Тёмно-синий", val: "#16203a" },
  { name: "Голубой", val: "#1f8fb0" },
  { name: "Зелёный", val: "#1f9d4d" },
  { name: "Тёмно-зелёный", val: "#14532d" },
  { name: "Красный", val: "#c62f2f" },
  { name: "Бордовый", val: "#7a1f2b" },
  { name: "Фиолетовый", val: "#6d28d9" },

```

```

    { name: "Светло-коричневый", val: "#8a5a2b" },
];

/* --- тихая попытка спросить Claude (не блокирует, без ошибок) --- */
async function callClaude(prompt) {
    const res = await fetch("https://api.anthropic.com/v1/messages", {
        method: "POST",
        headers: { "Content-Type": "application/json", "anthropic-version": "2023-06-
        body: JSON.stringify({ model: "claude-sonnet-4-6", max_tokens: 1000, messages
    });
    const data = await res.json();
    if (data && data.error) throw new Error("api");
    let text = (data.content || []).filter((b) => b.type === "text").map((b) => b.t
    text = text.replace(/``json/gi, "").replace(/``/g, "");
    const s = text.indexOf("[");
    let body = s >= 0 ? text.slice(s) : text;
    const e = body.lastIndexOf("]");
    if (e >= 0) body = body.slice(0, e + 1);
    body = body.replace(/,\s*([\]}])/g, "$1");
    try {
        return JSON.parse(body);
    } catch (_) {
        const objs = [...text.matchAll(/\{[^\}]*\}/g)].map((m) => {
            try { return JSON.parse(m[0].replace(/,\s*/, " ")); } catch { return null;
        }).filter(Boolean);
        if (objs.length) return objs;
        throw new Error("parse");
    }
}

async function fetchAI({ count, specs, difficulty }) {
    const [mn, mx] = RANGES[difficulty];
    const lines = specs.map((s) => s.subs.length ? `${s.theme} (${s.subs.join(", ")
    const prompt =
        `Составь ${count + 4} русских слов по темам: ${lines}. Уровень — ${LEVEL_RU[d
        `Подсказка до 8 слов, без самого слова. Существительные, им. падеж, ед. число
        `Верни ТОЛЬКО JSON: [{"word":"СЛОВО","clue":"подсказка"}]`;
    const arr = await callClaude(prompt);
    const seen = new Set(), out = [];
    if (Array.isArray(arr)) {
        for (const it of arr) {
            const w = String(it.word || "").toUpperCase().replace(/[\p{L}]/gu, "");
            if (w.length < mn || w.length > mx || seen.has(w)) continue;
            seen.add(w);
            out.push({ word: w, clue: String(it.clue || "").trim() });
        }
    }
}

```

```

    return out;
}

function buildSpecs(themes, subsSel, customSubs) {
    return themes.map((th) => {
        const pre = subsSel.filter((k) => k.startsWith(th + ">")).map((k) => k.split(
        const cs = customSubs[th] || [];
        let subs = [...pre, ...cs];
        if (!subs.length && THEMES[th]) subs = shuffle(THEMES[th]).slice(0, 2);
        return { theme: th, subs };
    });
}

const timeoutPromise = (ms) => new Promise((_, rej) => setTimeout(() => rej(new E

/* --- сборка отдельного HTML для печати / PDF --- */
function gridTableHTML(puzzle, ans) {
    let h = '<table class="cw">';
    for (let r = 0; r < puzzle.rows; r++) {
        h += "<tr>";
        for (let c = 0; c < puzzle.cols; c++) {
            const v = puzzle.sol[r][c];
            if (v == null) { h += '<td class="b"></td>'; }
            else {
                const n = puzzle.numbers[r][c];
                h += '<td class="c">' + (n > 0 ? "<i>" + n + "</i>" : "") + (ans ? v : ""
            }
        }
        h += "</tr>";
    }
    return h + "</table>";
}

function cluesColsHTML(puzzle, ans) {
    const mk = (title, list) => '<div><h3>' + title + '</h3>' +
    list.map((it) => '<p><b>' + it.num + '.</b>' + (ans ? it.answer : (it.clue |
    return '<div class="cl">' + mk("По горизонтали", puzzle.across) + mk("По вертик
}

function buildPrintHTML(puzzle, themes, withAnswers) {
    const css = "body{font-family:Arial,Helvetica,sans-serif;color:#000;margin:16px
    "h1{font-size:20px;margin:0 0 4px;}.sub{color:#555;font-size:12px;margin-bott
    "table.cw{border-collapse:collapse;}td{width:26px;height:26px;border:1px soli
    "td.b{background:#000;}td.c i{position:absolute;top:0;left:1px;font-size:8px;
    ".cl{display:flex;gap:28px;margin-top:14px;font-size:12.5px;}.cl h3{font-size
    ".pg{page-break-before:always;margin-top:24px;}@media print{button{display:no
    let body = "<h1>Кроссворд</h1><div class=\"sub\">" + themes.join(" · ") + "</di
    gridTableHTML(puzzle, false) + cluesColsHTML(puzzle, false);
    if (withAnswers) {

```



```

    body += '<div class="pg"><h1>Ответы</h1>' + gridTableHTML(puzzle, true) + clu
  }
  return "<!doctype html><html><head><meta charset=\"utf-8\"><title>Кроссворд</ti
    \"</style></head><body>" + body +
    "<script>window.onload=function(){setTimeout(function(){window.print();},400)
  }

```

/* --- рендер кроссворда в картинку (заливка выбранным цветом) --- */

```

function renderCanvas(puzzle, themes, withAnswers, ink, accent, gold) {
  ink = ink || "#16203a"; accent = accent || "#b81d1d"; gold = gold || "#c2a14a";
  const scale = 2, pad = 24, W = 820;
  const cols = puzzle.cols, rows = puzzle.rows;
  const cell = Math.max(18, Math.min(40, Math.floor((W - 2 * pad) / cols)));
  const gridH = rows * cell;
  const colW = (W - 2 * pad - 24) / 2;
  const m = document.createElement("canvas").getContext("2d");
  m.font = "13px Arial";
  const wrap = (text, maxW) => {
    const words = String(text).split(" "), lines = [];
    let cur = "";
    for (const w of words) {
      const t = cur ? cur + " " + w : w;
      if (m.measureText(t).width > maxW && cur) { lines.push(cur); cur = w; } else
    }
    if (cur) lines.push(cur);
    return lines;
  };
  const block = (list, ans) => list.map((it) => wrap(it.num + ". " + (ans ? it.an
  const lineH = 18, headH = 22, gap = 4;
  const colH = (items) => headH + items.reduce((s, l) => s + l.length * lineH + g
  const aA = block(puzzle.across, false), dA = block(puzzle.down, false);
  const cluesH = Math.max(colH(aA), colH(dA));
  let aB = null, dB = null, cluesH2 = 0;
  if (withAnswers) { aB = block(puzzle.across, true); dB = block(puzzle.down, tru
  const sec1 = gridH + 16 + cluesH + 20;
  const sec2 = withAnswers ? (30 + gridH + 16 + cluesH2 + 20) : 0;
  const totalH = pad + 26 + 18 + sec1 + sec2 + pad;

  const canvas = document.createElement("canvas");
  canvas.width = W * scale; canvas.height = totalH * scale;
  const ctx = canvas.getContext("2d");
  ctx.scale(scale, scale);
  ctx.fillStyle = "#fff"; ctx.fillRect(0, 0, W, totalH);
  ctx.textBaseline = "top"; ctx.textAlign = "left";

  let y = pad;

```

```

ctx.fillStyle = "#000"; ctx.font = "700 20px Arial"; ctx.fillText("Кроссворд",
ctx.fillStyle = "#555"; ctx.font = "12px Arial"; ctx.fillText(themes.join(" · "

const drawSection = (aItems, dItems, ans) => {
  const gy = y;
  for (let r = 0; r < rows; r++) for (let c = 0; c < cols; c++) {
    const x = pad + c * cell, yy = gy + r * cell, v = puzzle.sol[r][c];
    if (v == null) { ctx.fillStyle = ink; ctx.fillRect(x, yy, cell, cell); }
    else {
      ctx.fillStyle = "#fff"; ctx.fillRect(x, yy, cell, cell);
      ctx.strokeStyle = "#222"; ctx.lineWidth = 1; ctx.strokeRect(x + 0.5, yy +
      const n = puzzle.numbers[r][c];
      if (n > 0) { ctx.fillStyle = "#000"; ctx.font = "700 9px Arial"; ctx.text
      if (ans) { ctx.fillStyle = ink; ctx.font = Math.floor(cell * 0.6) + "px G
    }
  }
  ctx.strokeStyle = gold; ctx.lineWidth = 6; ctx.strokeRect(pad - 4, gy - 4, co
  ctx.strokeStyle = accent; ctx.lineWidth = 4; ctx.strokeRect(pad - 1, gy - 1,
  ctx.textAlign = "left"; ctx.textBaseline = "top";
  const startY = gy + gridH + 16;
  const drawCol = (x, items, header) => {
    let cy = startY;
    ctx.fillStyle = "#000"; ctx.font = "700 13px Arial"; ctx.fillText(header, x
    ctx.font = "13px Arial";
    items.forEach((lines) => { lines.forEach((ln) => { ctx.fillStyle = "#000";
    return cy;
  });
  const c1 = drawCol(pad, aItems, "По горизонтали");
  const c2 = drawCol(pad + colW + 24, dItems, "По вертикали");
  y = Math.max(c1, c2) + 20;
};

drawSection(aA, dA, false);
if (withAnswers) {
  ctx.fillStyle = "#000"; ctx.font = "700 18px Arial"; ctx.fillText("Ответы", p
  drawSection(aB, dB, true);
}
return canvas;
}

/* ===== ПАЛИТРА ===== */

const STYLES = [
  { key: "cl", label: "Классический", paper: "#f0f0f0", card: "#ffffff", soft: "#
  { key: "cn", label: "Китайский", paper: "#efe4c9", card: "#fbf6ea", soft: "#8a7
  { key: "jp", label: "Японский", paper: "#f5eef0", card: "#fffafb", soft: "#8a6f
  { key: "eu", label: "Европейский", paper: "#efeadb", card: "#fbf9f1", soft: "#6

```

```

    { key: "sl", label: "Русский", paper: "#f6f5f1", card: "#ffffff", soft: "#5a648"
    { key: "af", label: "Африканский", paper: "#eee3c8", card: "#fbf4e2", soft: "#5
];

```

```

function PrintGrid({ puzzle, answers }) {
  const mm = 7;
  return (
    <div style={{ display: "grid", gridTemplateColumns: `repeat(${puzzle.cols}, $
      {puzzle.sol.map((row, r) => row.map((val, c) => {
        const key = r + "-" + c;
        if (val == null) return <div key={key} style={{ width: `${mm}mm`, height:
        return (
          <div key={key} style={{ position: "relative", width: `${mm}mm`, height:
            {puzzle.numbers[r][c] > 0 && <span style={{ position: "absolute", top
              {answers ? val : ""}
            </div>
          </div>
        );
      })))}
    </div>
  );
}

```

```

/* ===== КОМПОНЕНТ ===== */

```

```

export default function CrosswordApp() {
  const [difficulty, setDifficulty] = useState("medium");
  const [themesSel, setThemesSel] = useState([]);
  const [customThemes, setCustomThemes] = useState([]);
  const [customThemeInput, setCustomThemeInput] = useState("");
  const [subsSel, setSubsSel] = useState([]);
  const [customSubs, setCustomSubs] = useState({});
  const [customSubInput, setCustomSubInput] = useState({});
  const [inkColor, setInkColor] = useState("#1c1c1c");
  const [styleIdx, setStyleIdx] = useState(0);
  const [loading, setLoading] = useState(false);
  const [source, setSource] = useState("");
  const [rawText, setRawText] = useState("");
  const [printAnswers, setPrintAnswers] = useState(false);
  const [panel, setPanel] = useState(null);
  const [printPreview, setPrintPreview] = useState(false);
  const [previewImg, setPreviewImg] = useState("");

  const [puzzle, setPuzzle] = useState(null);
  const [usedThemes, setUsedThemes] = useState([]);
  const [user, setUser] = useState([]);
  const [active, setActive] = useState(null);
  const [dir, setDir] = useState("A");

```

```

const [checked, setChecked] = useState(false);
const [wordModal, setWordModal] = useState(null);
const [wordThemes, setWordThemes] = useState({});
const [winW, setWinW] = useState(typeof window !== "undefined" ? window.innerWidth);
const inputRefs = useRef({});
const modalRefs = useRef({});
const gridRef = useRef(null);
const apiBroken = useRef(false);

const ST = STYLES[styleIdx] || STYLES[0];
const C = { paper: ST.paper, card: ST.card, ink: "#16203a", inkSoft: ST.soft, 1
const GOLD = ST.gold;
const REDX = ST.accent;
const REDX2 = ST.accent2 || ST.accent;
const T = { ...C, ink: inkColor, block: inkColor };

useEffect(() => {
  const onR = () => setWinW(window.innerWidth);
  window.addEventListener("resize", onR);
  return () => window.removeEventListener("resize", onR);
}, []);

const buildFrom = useCallback((entries) => {
  const p = buildCrossword(entries);
  setPuzzle(p);
  setChecked(false);
  setActive(null);
  if (p) setUser(p.sol.map((row) => row.map((x) => (x == null ? null : ""))));
}, []);

const generate = useCallback(async () => {
  setLoading(true);
  const count = WORD_COUNT[difficulty];
  let themes = [...themesSel];
  customThemes.forEach((t) => { if (t.trim()) themes.push(t.trim()); });
  if (!themes.length) themes = pickThemes(THEME_COUNT[difficulty]);
  setUsedThemes(themes);

  const hasCustom = customThemes.length > 0 || Object.values(customSubs).some((
  let words = bankPool(themes).slice(0, count + 12);
  let src = words.length ? "Встроенная база" : "";

  if (hasCustom || !apiBroken.current) {
    try {
      const specs = buildSpecs(themes, subsSel, customSubs);
      const ai = await Promise.race([fetchAI({ count, specs, difficulty }), tim
      if (ai && ai.length >= Math.max(6, Math.floor(count * 0.7))) { words = ai

```

```

        else if (!words.length && ai && ai.length) { words = ai.map((w) => ({ ...
    } catch (e) {
        apiBroken.current = true;
    }
}
if (words.length < count) {
    const have = new Set(words.map((w) => w.word));
    for (const w of bankPool(THEME_NAMES)) {
        if (words.length >= count + 12) break;
        if (!have.has(w.word)) { have.add(w.word); words.push(w); }
    }
}
if (words.length < 4) words = bankPool(THEME_NAMES).slice(0, count + 12);

setSource(src || "встроенная база");
const tmap = {};
words.forEach((w) => { if (!tmap[w.word]) tmap[w.word] = w.theme || "-"; });
setWordThemes(tmap);
setRawText(words.map((w) => `${w.word} - ${w.clue}`).join("\n"));
buildFrom(words.slice(0, count + 8));
setLoading(false);
}, [difficulty, themesSel, customThemes, subsSel, customSubs, buildFrom]);

const didInit = useRef(false);
useEffect(() => { if (didInit.current) return; didInit.current = true; generate

const cellSize = useMemo(() => {
    if (!puzzle) return 30;
    const avail = Math.min(winW, 600) - 36;
    return Math.max(18, Math.min(40, Math.floor(avail / puzzle.cols)));
}, [puzzle, winW]);

const activeWord = useMemo(() => (!puzzle || !active ? new Set() : new Set(word
const focusCell = (r, c) => { const el = inputRefs.current[r + "-" + c]; if (el

const step = (r, c, d, back) => {
    if (!puzzle) return;
    const dr = (d === "D" ? 1 : 0) * (back ? -1 : 1), dc = (d === "A" ? 1 : 0) *
    const rr = r + dr, cc = c + dc;
    if (puzzle.sol[rr] && puzzle.sol[rr][cc] !== null) { setActive({ r: rr, c: cc
};
const handleChange = (r, c, val) => {
    const ch = (val || "").slice(-1).toUpperCase();
    if (ch && !/[p{L}\p{N}]/u.test(ch)) return;
    setUser((prev) => { const g = prev.map((row) => row.slice()); g[r][c] = ch; r
    if (ch) step(r, c, dir, false);
};

```

```

const toggleDir = (r, c) => { const o = dir === "A" ? "D" : "A"; if (hasWord(pu
const handleKey = (e, r, c) => {
  if (!puzzle) return;
  const k = e.key;
  if (k === "ArrowRight") { e.preventDefault(); setDir("A"); step(r, c, "A", fa
  else if (k === "ArrowLeft") { e.preventDefault(); setDir("A"); step(r, c, "A"
  else if (k === "ArrowDown") { e.preventDefault(); setDir("D"); step(r, c, "D"
  else if (k === "ArrowUp") { e.preventDefault(); setDir("D"); step(r, c, "D",
  else if (k === "Backspace") { if (!user[r][c]) { e.preventDefault(); step(r,
  else if (k === " ") { e.preventDefault(); toggleDir(r, c); }
};

```

```

const clueFor = (r, c, d) => {
  const list = d === "A" ? puzzle.across : puzzle.down;
  const cells = wordCells(puzzle.sol, r, c, d);
  if (!cells.length) return null;
  const [sr, sc] = cells[0].split("-").map(Number);
  return list.find((it) => it.r === sr && it.c === sc) || null;
};

```

```

const openWordAt = (r, c, d) => { const it = clueFor(r, c, d); if (it) setWordM
const selectCell = (r, c) => {
  let d = dir;
  if (active && active.r === r && active.c === c) {
    d = dir === "A" ? "D" : "A";
    if (!hasWord(puzzle.sol, r, c, d)) d = dir;
  } else if (!hasWord(puzzle.sol, r, c, d)) {
    d = d === "A" ? "D" : "A";
  }
  setDir(d);
  setActive({ r, c });
  openWordAt(r, c, d);
};

```

```

const reveal = () => { if (puzzle) { setUser(puzzle.sol.map((row) => row.map((x
const clearAll = () => { if (puzzle) { setUser(puzzle.sol.map((row) => row.map(
const play = () => {
  if (!puzzle) return;
  const it = puzzle.across[0] || puzzle.down[0];
  if (it) { const d = puzzle.across[0] ? "A" : "D"; setDir(d); setWordModal({ d
};

```

```

const sharePuzzle = () => {
  try {
    const canvas = renderCanvas(puzzle, usedThemes, printAnswers, inkColor, RED
    canvas.toBlob(async (blob) => {
      if (!blob) return;
      const file = new File([blob], "crossgen.png", { type: "image/png" });

```

```

    try {
      if (navigator.canShare && navigator.canShare({ files: [file] })) {
        await navigator.share({ files: [file], title: "Кроссворд", text: "Кро
        return;
      }
    } catch (e) {}
    try {
      const url = URL.createObjectURL(blob);
      const a = document.createElement("a");
      a.href = url; a.download = "crossgen.png";
      document.body.appendChild(a); a.click(); a.remove();
    } catch (e) {}
  }, "image/png");
} catch (e) {}
};

```

```

const openPrintWindow = () => {
  const html = buildPrintHTML(puzzle, usedThemes, printAnswers);
  try {
    const w = window.open("", "_blank");
    if (w) { w.document.open(); w.document.write(html); w.document.close(); ret
  } catch (e) {}
  try {
    const url = URL.createObjectURL(new Blob([html], { type: "text/html" }));
    const a = document.createElement("a");
    a.href = url; a.target = "_blank"; a.rel = "noopener";
    document.body.appendChild(a); a.click(); a.remove();
  } catch (e) {}
};

```

```

const stat = useMemo(() => {
  if (!puzzle || !user.length) return { filled: 0, total: 0, correct: 0 };
  let filled = 0, total = 0, correct = 0;
  for (let r = 0; r < puzzle.rows; r++) {
    for (let c = 0; c < puzzle.cols; c++) {
      if (puzzle.sol[r][c] == null) continue;
      total++;
      if (user[r] && user[r][c]) filled++;
      if (user[r] && user[r][c] === puzzle.sol[r][c]) correct++;
    }
  }
  return { filled, total, correct };
}, [puzzle, user]);
const allDone = puzzle && stat.total > 0 && stat.correct === stat.total;

```

```

const composition = useMemo(() => {
  if (!puzzle || !Object.keys(wordThemes).length) return [];

```

```

const counts = {}, seen = new Set();
[...puzzle.across, ...puzzle.down].forEach((it) => {
  if (seen.has(it.answer)) return;
  seen.add(it.answer);
  const th = wordThemes[it.answer] || "-";
  counts[th] = (counts[th] || 0) + 1;
});
return Object.entries(counts).sort((a, b) => b[1] - a[1]);
}, [puzzle, wordThemes]);
const missingThemes = useMemo(() => {
  if (!composition.length) return [];
  const inGrid = new Set(composition.map((c) => c[0]));
  return usedThemes.filter((t) => !inGrid.has(t));
}, [composition, usedThemes]);

const toggleTheme = (t) => {
  const removing = themesSel.includes(t);
  setThemesSel((s) => (removing ? s.filter((x) => x !== t) : [...s, t]));
  if (removing) {
    setSubsSel((s) => s.filter((k) => !k.startsWith(t + ">")));
    setCustomSubs((cs) => { const n = { ...cs }; delete n[t]; return n; });
  }
};

const toggleSub = (key) => setSubsSel((s) => (s.includes(key) ? s.filter((x) =>
const addCustomTheme = () => {
  const v = customThemeInput.trim();
  if (!v) return;
  if (![...themesSel, ...customThemes, ...THEME_NAMES].some((x) => x.toLowerCase() === v.toLowerCase()))
    setCustomThemes((a) => [...a, v]);
}
setCustomThemeInput("");
};

const removeCustomTheme = (t) => {
  setCustomThemes((a) => a.filter((x) => x !== t));
  setSubsSel((s) => s.filter((k) => !k.startsWith(t + ">")));
  setCustomSubs((cs) => { const n = { ...cs }; delete n[t]; return n; });
};

const addCustomSub = (th) => {
  const v = (customSubInput[th] || "").trim();
  if (!v) return;
  setCustomSubs((cs) => {
    const cur = cs[th] || [];
    if (cur.some((x) => x.toLowerCase() === v.toLowerCase())) return cs;
    return { ...cs, [th]: [...cur, v] };
  });
  setCustomSubInput((m) => ({ ...m, [th]: "" }));
};

```



```

const removeCustomSub = (th, s) => setCustomSubs((cs) => ({ ...cs, [th]: (cs[th]
const selectedThemes = [...themesSel, ...customThemes];

const btn = (bg, color, extra) => ({ background: bg, color, border: "none", bor
const seg = (val) => ({ flex: 1, padding: "9px 0", border: "none", borderRadius
const chip = (on) => ({ border: on ? `1.5px solid ${REDX}` : `1.5px solid ${T.l
const out = btn(T.card, REDX, { boxShadow: `inset 0 0 0 1.5px ${T.line}` });
const recTheme = THEME_COUNT[difficulty];

const modalData = useMemo(() => {
  if (!puzzle || !wordModal) return null;
  const { dir: d, item } = wordModal;
  const cells = wordCells(puzzle.sol, item.r, item.c, d);
  return { cells, item, dir: d };
}, [puzzle, wordModal]);

const modalChange = (cells, idx, val) => {
  const ch = (val || "").slice(-1).toUpperCase();
  if (ch && !/[p{L}]/u.test(ch)) return;
  const [r, c] = cells[idx].split("-").map(Number);
  setUser((prev) => { const g = prev.map((row) => row.slice()); g[r][c] = ch; r
  if (ch && idx + 1 < cells.length) { const el = modalRefs.current[idx + 1]; if
};

const modalKey = (e, cells, idx) => {
  const [r, c] = cells[idx].split("-").map(Number);
  if (e.key === "Backspace" && !(user[r] && user[r][c]) && idx > 0) {
    e.preventDefault();
    const el = modalRefs.current[idx - 1];
    if (el) el.focus();
  }
};

const nextWord = () => {
  if (!puzzle || !wordModal) return;
  const order = [...puzzle.across.map((i) => ({ dir: "A", item: i })), ...puzzl
  const idx = order.findIndex((o) => o.dir === wordModal.dir && o.item.num ===
  const nx = order[(idx + 1) % order.length];
  setDir(nx.dir);
  setWordModal(nx);
};

return (
  <div style={{ minHeight: "100vh", background: T.paper, fontFamily: "Inter, sa
    <style>{`
      @import url('https://fonts.googleapis.com/css2?family=Unbounded:wght@600;
      * { box-sizing: border-box; -webkit-tap-highlight-color: transparent; }
      textarea:focus, input:focus { outline: 2px solid ${T.ink}; outline-offset
      .cwcell input:focus, .mbox input:focus { outline: none; }

```

```

.clueRow:hover { background: ${T.word}66; }
@keyframes spin { to { transform: rotate(360deg); } }
.spin { animation: spin .8s linear infinite; }
.cwcell { transition: background .12s ease; }
`}</style>

<div className="screen" style={{ maxWidth: 600, margin: "0 auto", padding:
  <div style={{ display: "flex", alignItems: "center", gap: 12 }}>
    <div style={{ width: 48, height: 48, borderRadius: 8, background: ST.se
      <div>
        <div style={{ fontFamily: ST.titleFont, fontWeight: 400, fontSize: 33
          <div style={{ fontFamily: ST.titleFont, fontWeight: 400, fontSize: 33
            </div>
          </div>
        </div>
      </div>

    <div style={{ background: T.card, borderRadius: 16, padding: 16, marginTo
      <div style={{ fontSize: 13.5, fontWeight: 600, marginBottom: 8, color:
        <div style={{ display: "flex", flexWrap: "wrap", gap: 5, marginBottom:
          {STYLES.map((s, i) => (
            <button key={s.key} onClick={() => { setStyleIdx(i); setInkColor(ST
              )}}
            </div>
          <div style={{ fontSize: 13.5, fontWeight: 600, marginBottom: 8, color:
            <div style={{ display: "flex", gap: 4, background: "#f1f3f7", borderRad
              <button style={seg("easy")} onClick={() => setDifficulty("easy")}>Лёг
              <button style={seg("medium")} onClick={() => setDifficulty("medium")}
              <button style={seg("hard")} onClick={() => setDifficulty("hard")}>Сло
            </div>
          </div>

        <div style={{ display: "flex", justifyContent: "space-between", alignIt
          <span style={{ fontSize: 13.5, fontWeight: 600, color: REDX2 }}>Темы<
          <span style={{ fontSize: 12, color: T.inkSoft }}>выбрано {selectedThe
        </div>

        <div style={{ display: "flex", flexWrap: "wrap", gap: 3 }}>
          {THEME_NAMES.map((t) => <button key={t} onClick={() => toggleTheme(t)
            {customThemes.map((t) => <button key={"c" + t} onClick={() => removeC
          </div>

        <div style={{ display: "flex", gap: 6, marginTop: 6 }}>
          <input value={customThemeInput} onChange={(e) => setCustomThemeInput(
            <button onClick={addCustomTheme} style={btn(REDX, "#fff", { padding:
          </div>

        {selectedThemes.length > 0 && (
          <div style={{ marginTop: 16 }}>
            <div style={{ fontSize: 13.5, fontWeight: 600, marginBottom: 4, col
              {selectedThemes.map((th) => (
                <div key={th} style={{ marginTop: 10 }}>

```

```

        <div style={{ fontSize: 12, fontWeight: 600, color: T.inkSoft,
        <div style={{ display: "flex", flexWrap: "wrap", gap: 3, alignI
            {(THEMES[th] || []).map((sub) => {
                const key = th + ">" + sub;
                return <button key={key} onClick={() => toggleSub(key)} sty
            }}}
            {(customSubs[th] || []).map((s) => <button key={"cs" + s} onC
            <input value={customSubInput[th] || ""} onChange={(e) => setC
            <button onClick={() => addCustomSub(th)} style={btn(REDX, "#f
        </div>
    </div>
    )))
</div>
))
{selectedThemes.length === 0 && <div style={{ fontSize: 11.5, color: T.
{customThemes.length > 0 && <div style={{ fontSize: 11.5, color: T.inkS

    <button onClick={generate} disabled={loading} style={btn(REDX, "#fff",
        {loading && <span className="spin" style={{ width: 15, height: 15, bo
        {loading ? "Собираю кроссворд..." : "Сгенерировать кроссворд"}
    </button>
</div>

{puzzle && (
    <div style={{ display: "flex", gap: 14, alignItems: "center", margin: "
        <span><b style={{ color: T.ink }}>{puzzle.total - (puzzle.unplaced?.1
        <span>заполнено <b style={{ color: T.ink }}>{stat.filled}</b>/{stat.t
        {checked && <><span>·</span><span style={{ color: stat.correct === st
    </div>
))
{puzzle && <div style={{ fontSize: 12, color: T.inkSoft, margin: "0 2px 1
{puzzle && composition.length > 0 && (
    <div style={{ fontSize: 12, color: T.inkSoft, margin: "-6px 2px 12px",
        <b style={{ color: REDX2 }}>Состав:</b> {composition.map(([t, n]) =>
        {missingThemes.length > 0 && <span style={{ color: C.wrong }}> · не в
    </div>
))
{allDone && <div style={{ background: C.correct, color: "#fff", borderRad

{puzzle && (
    <div ref={gridRef} style={{ overflowX: "auto", padding: "4px 2px 2px",
        <div style={{ display: "grid", gridTemplateColumns: `repeat(${puzzle.
            {puzzle.sol.map((row, r) => row.map((val, c) => {
                const key = r + "-" + c;
                if (val == null) return <div key={key} style={{ width: cellSize,
                const inWord = activeWord.has(key);
                const isCell = active && active.r === r && active.c === c;

```

```

    const uv = user[r] ? user[r][c] : "";
    let bg = T.card;
    if (inWord) bg = T.word;
    if (isCell) bg = T.cell;
    let tc = T.ink;
    if (checked && uv) tc = uv === val ? C.correct : C.wrong;
    return (
      <div key={key} className="cwcell" style={{ position: "relative"
        {puzzle.numbers[r][c] > 0 && <span style={{ position: "absolu
          <input ref={el} => (inputRefs.current[key] = el)} value={uv
        </div>
      });
    })))
  </div>
</div>
)}

{puzzle && (
  <>
    <div style={{ display: "flex", gap: 10, marginTop: 16 }}>
      <button onClick={play} style={btn(REDX, "#fff", { flex: 2, fontSize
        <button onClick={() => setPanel(panel === "print" ? null : "print")}
        <button onClick={() => setPanel(panel === "edit" ? null : "edit")}
      </div>

      {panel === "print" && (
        <div style={{ background: T.card, borderRadius: 14, padding: 14, ma
          <label style={{ display: "flex", alignItems: "center", gap: 8, fo
            <input type="checkbox" checked={printAnswers} onChange={(e) =>
              Печатать ответы (отдельная страница)
          </label>
          <button onClick={() => { try { const cv = renderCanvas(puzzle, us
        </div>
      )}

      {panel === "edit" && (
        <div style={{ background: T.card, borderRadius: 14, padding: 14, ma
          <div style={{ display: "flex", gap: 10, flexWrap: "wrap" }}>
            <button onClick={() => setChecked(true)} style={{ ...out, flex:
            <button onClick={reveal} style={{ ...out, flex: "1 1 100px" }}>
            <button onClick={clearAll} style={{ ...out, flex: "1 1 100px" }
          </div>
          <div style={{ fontSize: 12.5, color: T.inkSoft, margin: "14px 0 6
            <textarea value={rawText} onChange={(e) => setRawText(e.target.va
            <button onClick={() => buildFrom(parseEntries(rawText))} style={b
          </div>
        )}

```

```

    </>
  )}

{puzzle && (
  <div style={{ display: "grid", gridTemplateColumns: winW > 520 ? "1fr 1
    {[["По горизонтали", puzzle.across, "A"], ["По вертикали", puzzle.dow
    <div key={d}>
      <div style={{ fontFamily: ST.titleFont, fontWeight: 400, fontSize
        {list.map((item) => {
          const sel = active && item.r === active.r && item.c === active.
          return (
            <div key={d + item.num} className="clueRow" onClick={() => {
              <span style={{ fontFamily: "'IBM Plex Mono', monospace", fo
              <span style={{ fontSize: 13.5, lineHeight: 1.45, color: T.i
            </div>
          );
        }}}
      </div>
    )}}
  </div>
)}
{puzzle && <div style={{ textAlign: "center", marginTop: 28, fontSize: 12
</div>

{/* OKHO CJOBA */}
{modalData && (
  <div onClick={() => setWordModal(null)} style={{ position: "fixed", inset
    <div onClick={(e) => e.stopPropagation()} style={{ background: T.card,
      <div style={{ display: "flex", justifyContent: "space-between", align
        <span style={{ fontFamily: "'IBM Plex Mono', monospace", fontSize:
          <button onClick={() => setWordModal(null)} style={{ border: "none",
        </div>
      <div style={{ fontSize: 17, fontWeight: 600, margin: "8px 0 16px", li
      <div className="mbox" style={{ display: "flex", gap: 6, flexWrap: "wr
        {modalData.cells.map((key, idx) => {
          const [r, c] = key.split("-").map(Number);
          const uv = user[r] ? user[r][c] : "";
          const ok = checked && uv && uv === puzzle.sol[r][c];
          const bad = checked && uv && uv !== puzzle.sol[r][c];
          return (
            <div key={key} style={{ position: "relative", width: 42, height
              <input ref={(el) => (modalRefs.current[idx] = el)} value={uv
                {checked && (ok || bad) && <span style={{ position: "absolute
              </div>
            );
          }}}
        </div>
      )}
    </div>
  )}

```

```

        <div style={{ display: "flex", gap: 10, marginTop: 18 }}>
            <button onClick={() => setChecked(true)} style={{ ...out, flex: 1 }}
            <button onClick={nextWord} style={btn(REDX, "#fff", { flex: 1.4 })}
        </div>
    </div>
</div>
))

{/* ПРЕДПРОСМОТР ПЕЧАТИ */}
{printPreview && puzzle && (
    <div style={{ position: "fixed", inset: 0, background: "#fff", zIndex: 60
    <div style={{ display: "flex", gap: 8, alignItems: "center", padding: "
        <span style={{ fontFamily: "Unbounded, sans-serif", fontWeight: 700,
        <button onClick={sharePuzzle} style={btn(REDX, "#fff", { padding: "9p
        <button onClick={() => setPrintPreview(false)} style={btn(T.card, T.i
    </div>
    <div style={{ overflow: "auto", padding: 16 }}>
        <div style={{ background: T.word, border: `1px solid ${C.cell}`, bord
        {previewImg
            ? <img src={previewImg} alt="Кроссворд" style={{ width: "100%", bor
            : <div style={{ color: T.inkSoft }}>Готовлю изображение...</div>}
        </div>
    </div>
    )}

</div>
);
}

```